



Electrónica Digital III

Avance del Proyecto Final

SmartRoad Rover

Juan Felipe Orozco Londoño

Universidad de Antioquia

25 de mayo de 2026

Índice general

1. Plan de Trabajo	3
1.1. Descripción general	3
1.2. Desglose de tareas y avance	3
1.3. Resumen de avance global	5
2. Cambios Respecto a la Definición — Ronda 2	6
2.1. Lenguaje de programación: MicroPython → C / Pico SDK	6
2.2. Implementación física: PCB → Protoboard	6
2.3. Actualización del presupuesto	7
2.4. Correcciones de la Ronda 2 incorporadas	7
3. Circuitos Analógicos Requeridos	9
3.1. Lista de circuitos requeridos	9
3.2. Circuitos 1, 2 y 3 — Divisor de tensión resistivo (HC-SR04 ECHO)	9
3.2.1. Problema que resuelve	9
3.2.2. Topología seleccionada	10
3.2.3. Procedimiento de diseño y cálculo	10
3.2.4. Diagrama de conexiones en protoboard	10
3.2.5. Componentes necesarios por cada divisor ($\times 3$)	11
3.2.6. Estado de avance	11
3.3. Circuito 4 — Sistema de alimentación con regulador step-down	11
3.3.1. Problema que resuelve	11
3.3.2. Módulo seleccionado	11
3.3.3. Procedimiento de diseño y cálculo de corriente	12
3.3.4. Diagrama de conexiones en protoboard	12
3.3.5. Estado de avance	12
4. Diagrama de Bloques del Hardware	13
4.1. Decisión de diseño: un diagrama jerárquico en tres niveles	13
4.2. Asignación de pines GPIO del Pico W	13
4.3. Diagrama 1 — Vista general del sistema	15
4.4. Diagrama 2 — Bloque de sensado y acondicionamiento	16
4.5. Diagrama 3 — Bloque de actuación y sistema de alimentación	17

5. Arquitectura de Software	19
5.1. Visión general y decisiones de diseño	19
5.2. Estructura modular del firmware	19
5.3. Máquina de estados finita (FSM) — <code>fsm.c</code>	20
5.4. Módulo de sensores — ISR y filtrado (<code>sensors.c</code>)	23
5.5. Módulo de motores — PWM de hardware (<code>motors.c</code>)	24
5.6. Módulo de navegación autónoma (<code>navigation.c</code>)	25
5.7. Módulo GPS con DMA (<code>gps.c</code>) — Reporte DMA	27
5.8. Módulo servidor web — Watchdog WiFi (<code>web_server.c</code>)	28
5.9. Inventario de interrupciones	28
5.10. Librerías del Pico SDK utilizadas	29
6. Manejo del Repositorio	30
6.1. Repositorio del proyecto	30
6.2. Estructura de carpetas	30
6.3. Estrategia de ramas	31
6.4. Convención de mensajes de commit	31
6.5. Integración con VS Code y Pico SDK	32
6.6. Estado actual del repositorio	32

Capítulo 1

Plan de Trabajo

1.1. Descripción general

El proyecto SmartRoad Rover es desarrollado de forma individual por Juan Felipe Orozco Londoño. A continuación se presenta el plan de trabajo estructurado en 6 semanas, con las tareas definidas, su descripción y el estado de avance actual. El lenguaje de programación utilizado para el firmware es **C**, haciendo uso del **Pico SDK** oficial de Raspberry Pi, con una arquitectura basada en **polling e interrupciones (IRQ)**.

1.2. Desglose de tareas y avance

Semana 1 — Definición y diseño del sistema

100 %

- Definición de requisitos funcionales y no funcionales.
- Selección y justificación de componentes.
- Elaboración del diagrama de bloques del hardware.
- Estimación del presupuesto.
- Configuración del repositorio en GitHub con estructura de carpetas.

Semana 2 — Diseño detallado de hardware y arquitectura de software

100 %

- Diseño del esquemático del circuito (Pico W, HC-SR04, TB6612FNG, NEO-6M).
- Definición de conexiones y asignación de pines del microcontrolador.
- Diseño del prototipo sobre protoboard.
- Definición de la arquitectura de software en C/Pico SDK: módulos, máquina de estados, flujo de polling e interrupciones.
- Documentación del diseño en el repositorio.

Semana 3 — Adquisición de componentes y montaje del prototipo

60 %

- Adquisición de sensores HC-SR04, driver TB6612FNG, módulo GPS NEO-6M, chasis 2WD, batería 18650 y regulador step-down. (*En proceso*)
- Montaje del circuito sobre protoboard. (*Pendiente*)
- Verificación de alimentación y continuidad del circuito. (*Pendiente*)
- Prueba básica de encendido del sistema. (*Pendiente*)

Semana 4 — Desarrollo e integración del firmware

15 %

- Configuración del entorno de desarrollo Pico SDK en C.
- Implementación del módulo de lectura de sensores HC-SR04 mediante interrupciones (IRQ en flanco de subida y bajada).
- Implementación del módulo de control de motores DC vía PWM.
- Implementación del servidor web embebido para control manual vía WiFi.
- Implementación de la máquina de estados para gestión de modos de operación.
- Integración de módulos y pruebas unitarias.

Semana 5 — Integración completa y pruebas del sistema

0 %

- Integración del firmware completo sobre el prototipo ensamblado.
- Pruebas del modo autónomo reactivo (detección, evasión, tiempos de respuesta verificados por UART).
- Pruebas del modo manual vía WiFi.
- Pruebas del modo GPS en entorno exterior.
- Verificación de comportamientos de contingencia (pérdida de WiFi, pérdida de señal GPS).
- Corrección de errores y ajustes finos.

Semana 6 — Validación final y preparación de la entrega 0 %

- Pruebas completas en el escenario de pruebas definitivo (entorno exterior, 10 m × 10 m).
- Verificación de todos los requisitos funcionales (RF) y no funcionales (RNF).
- Actualización final del repositorio: firmware, hardware, documentación.
- Preparación de la sustentación del proyecto.

1.3. Resumen de avance global

Fase	Avance
Diseño y planificación del sistema	100 %
Diseño de hardware y esquemático	100 %
Adquisición de componentes y montaje	60 %
Desarrollo de firmware (C / Pico SDK)	15 %
Integración y pruebas	0 %
Validación final y entrega	0 %
Avance general estimado	≈45 %

Cuadro 1.1: Resumen del estado de avance del proyecto

Capítulo 2

Cambios Respecto a la Definición — Ronda 2

Se describen a continuación todos los cambios aplicados al proyecto respecto a la entrega de la Ronda 2, incluyendo correcciones derivadas de la retroalimentación del docente y decisiones técnicas tomadas durante la fase de diseño detallado.

2.1. Lenguaje de programación: MicroPython $\rightarrow$ C / Pico SDK

El firmware del sistema se implementará en **C** utilizando el **Pico SDK oficial** de Raspberry Pi. Esta decisión responde al requerimiento explícito del curso y es técnicamente más adecuada para una arquitectura basada en polling e interrupciones (IRQ), ya que el SDK en C ofrece acceso directo al hardware, control determinista de tiempos y manejo nativo de periféricos como PWM, UART y GPIO sin capas de abstracción intermedias.

2.2. Implementación física: PCB $\rightarrow$ Protoboard

La implementación física del circuito se realizará sobre **protoboard** en lugar de una placa de circuito impreso (PCB), por las siguientes razones:

- El tiempo disponible hasta la entrega final no es suficiente para completar el ciclo de diseño de layout, fabricación, recepción y soldadura de una PCB con garantías de calidad.
- La protoboard permite iteraciones rápidas durante el desarrollo y la depuración del firmware, facilitando modificaciones de conexiones sin riesgo de dañar un circuito permanente.
- Para un prototipo funcional de validación académica, la protoboard cumple completamente con todos los requisitos funcionales y no funcionales definidos.

El esquemático y el presupuesto de fabricación de PCB se conservan como documentación técnica, representando la hoja de ruta para un escalamiento futuro del proyecto. Como complemento, se contempla el diseño de una **carcasa en impresión 3D** para alojar el conjunto sobre el chasis 2WD, aportando robustez mecánica al prototipo.

2.3. Actualización del presupuesto

Como consecuencia del cambio de PCB a protoboard, el presupuesto se actualiza de la siguiente manera:

Categoría	Subtotal (COP)
Componentes electrónicos	126.000
Protopoard y materiales de conexión	25.000
Plataforma mecánica	53.000
Sistema de alimentación	48.000
Total estimado	252.000

Cuadro 2.1: Presupuesto actualizado — prototipo sobre protoboard

El total se reduce de 312.000 COP a **252.000 COP**, liberando recursos para la adquisición de componentes faltantes.

2.4. Correcciones de la Ronda 2 incorporadas

Los siguientes ajustes fueron aplicados con base en la retroalimentación del docente:

- **Nota administrativa:** se confirmó que no hubo cambio de nombre del proyecto ni novedades en el equipo de trabajo.
- **RNF reformulados:** el tiempo de respuesta se separó por modo de operación. El modo autónomo reactivo tiene un ciclo determinista menor a 200 ms; el modo manual reconoce que la latencia WiFi no es controlable por el microcontrolador.
- **RNF de tolerancia a fallos agregados:** RNF-5 (detención en ≤ 500 ms ante pérdida de WiFi) y RNF-6 (transición a modo autónomo reactivo ante pérdida de señal GPS por más de 3 segundos).
- **Escenario de pruebas unificado:** único entorno exterior de mínimo 10×10 m, materiales de obstáculos especificados (madera o cartón rígido) y verificación de tiempos por timestamps UART en lugar de cronómetro manual.
- **Radio GPS corregido:** reducido de 3 m a 1,5 m para ser coherente con el área de pruebas y la precisión real del módulo NEO-6M.

- **Costo de envío PCB incluido:** partida de envío internacional (~ 20.000 COP) incorporada al presupuesto de referencia para escalamiento futuro.

Capítulo 3

Circuitos Analógicos Requeridos

El sistema SmartRoad Rover requiere **cuatro circuitos analógicos** de acondicionamiento. Tres de ellos son idénticos (un divisor de tensión por cada sensor HC-SR04) y uno corresponde al sistema de alimentación con regulación de voltaje. A continuación se presenta la lista, el procedimiento de diseño de cada uno y su estado de avance.

3.1. Lista de circuitos requeridos

1. Divisor de tensión resistivo — sensor HC-SR04 frontal (ECHO: 5 V $\rightarrow$ 3.3 V)
2. Divisor de tensión resistivo — sensor HC-SR04 izquierdo (ECHO: 5 V $\rightarrow$ 3.3 V)
3. Divisor de tensión resistivo — sensor HC-SR04 derecho (ECHO: 5 V $\rightarrow$ 3.3 V)
4. Circuito de alimentación — regulador step-down 7.4 V $\rightarrow$ 5 V con dos dominios de voltaje

3.2. Circuitos 1, 2 y 3 — Divisor de tensión resistivo (HC-SR04 ECHO)

3.2.1. Problema que resuelve

El Raspberry Pi Pico W opera con lógica de **3.3 V**. Sus pines GPIO soportan un máximo de 3.3 V en la entrada. El sensor HC-SR04 opera a **5 V** y devuelve la señal ECHO a nivel de 5 V. Conectar directamente el pin ECHO al GPIO del Pico W daña el microcontrolador de forma permanente. Es necesario adaptar el nivel lógico de 5 V a 3.3 V antes de que la señal llegue al GPIO.

El pin TRIG del HC-SR04 acepta 3.3 V como nivel alto válido, por lo tanto la señal de salida del Pico W va directamente al TRIG sin circuito intermedio.

3.2.2. Topología seleccionada

Divisor resistivo de tensión. Es la solución más simple, económica y confiable para adaptación de nivel lógico unidireccional (el ECHO solo va del sensor al microcontrolador, no al revés). No requiere componentes activos ni encapsulados especiales.

3.2.3. Procedimiento de diseño y cálculo

La ecuación del divisor de tensión es:

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

Objetivo: $V_{in} = 5\text{ V}$, $V_{out} \leq 3,3\text{ V}$

Valores seleccionados: $R_1 = 1\text{ k}\Omega$, $R_2 = 2\text{ k}\Omega$

Verificación:

$$V_{out} = 5 \cdot \frac{2000}{1000 + 2000} = 5 \cdot \frac{2}{3} = 3,33\text{ V} \quad \checkmark$$

Este valor de 3.33 V es seguro para el GPIO del Pico W (máximo 3.3 V, con un margen de tolerancia del 1 %) y suficientemente alto para ser reconocido como nivel lógico HIGH (umbral típico $V_{IH} = 2,0\text{ V}$).

Corriente a través del divisor:

$$I = \frac{V_{in}}{R_1 + R_2} = \frac{5}{3000} \approx 1,67\text{ mA}$$

Valor despreciable, compatible con la capacidad de corriente del pin ECHO del HC-SR04.

3.2.4. Diagrama de conexiones en protoboard

Cada divisor de tensión se implementa del siguiente modo, replicado tres veces:

1. Pin **VCC** del HC-SR04 $\rightarrow$ riel de 5 V de la protoboard.
2. Pin **GND** del HC-SR04 $\rightarrow$ riel GND de la protoboard.
3. Pin **TRIG** del HC-SR04 $\rightarrow$ GPIO del Pico W (salida directa, sin circuito intermedio).
4. Pin **ECHO** del HC-SR04 $\rightarrow$ extremo de **R1 (1 k Ω)**.
5. Extremo libre de R1 $\rightarrow$ nodo intermedio en protoboard. Desde este mismo nodo:
 - Conectar **R2 (2 k Ω)** hacia GND.
 - Conectar un cable hacia el **pin GPIO del Pico W** (entrada ECHO_IN).

3.2.5. Componentes necesarios por cada divisor ($\times 3$)

Componente	Valor	Cantidad
Resistencia	1 k Ω / 0.25 W	1
Resistencia	2 k Ω / 0.25 W	1
Cable jumper macho-macho	—	4

Cuadro 3.1: Componentes por divisor de tensión (se requieren 3 conjuntos)

3.2.6. Estado de avance

Diseño: 100 %. Los valores están calculados, verificados y listos para implementar.
Implementación física: pendiente (sujeta a la adquisición de los sensores HC-SR04).

3.3. Circuito 4 — Sistema de alimentación con regulador step-down

3.3.1. Problema que resuelve

El sistema tiene **dos dominios de voltaje**:

- **7.4 V** (batería 18650): requerido directamente por los motores DC a través del pin VM del driver TB6612FNG.
- **5 V regulados**: requeridos por el Pico W (pin VSYS, acepta 1.8–5.5 V), los sensores HC-SR04 ($V_{cc} = 5$ V) y el módulo GPS NEO-6M ($V_{cc} = 3.3$ V, regulado internamente desde el pin 3V3OUT del Pico W).

No es posible alimentar directamente la lógica con 7.4 V sin dañar los componentes. Se requiere un regulador step-down (convertidor buck) que reduzca eficientemente 7.4 V a 5 V.

3.3.2. Módulo seleccionado

Módulo regulador step-down genérico basado en el integrado **MP1584** o similar. Parámetros relevantes:

- Voltaje de entrada: 4.5–28 V (compatible con 7.4 V de la batería).
- Voltaje de salida: ajustable; configurado a **5 V** mediante el potenciómetro del módulo.
- Corriente de salida máxima: 3 A.
- Eficiencia típica: 92 %.

3.3.3. Procedimiento de diseño y cálculo de corriente

Se estima la corriente total del dominio de 5 V:

Componente	Corriente típica
Raspberry Pi Pico W (WiFi activo)	100 mA
HC-SR04 $\times$ 3	15 mA $\times$ 3 = 45 mA
NEO-6M GPS	45 mA
Total dominio 5 V	190 mA

Cuadro 3.2: Consumo estimado del dominio de lógica

El regulador seleccionado (3 A) entrega un margen de seguridad mayor a 15:1 sobre la carga de lógica. Los motores se alimentan directamente desde la batería a través del TB6612FNG, sin pasar por el regulador.

3.3.4. Diagrama de conexiones en protoboard

1. **Batería (+)** $\rightarrow$ pin IN+ del módulo step-down y pin VM del TB6612FNG.
2. **Batería (-)** $\rightarrow$ riel GND común de la protoboard.
3. **Step-down OUT+** (5 V) $\rightarrow$ riel de 5 V de la protoboard.
4. **Riel 5 V** $\rightarrow$ pin VSYS del Pico W.
5. **Riel 5 V** $\rightarrow$ pin VCC de cada HC-SR04 ($\times$ 3).
6. **Pin 3V3OUT del Pico W** $\rightarrow$ pin VCC del módulo GPS NEO-6M.
7. **Riel GND común** $\rightarrow$ GND de todos los componentes del sistema.

Nota crítica: todos los componentes deben compartir el mismo GND (tierra común). Sin tierra común, las señales lógicas no tienen referencia y el sistema no funciona correctamente.

3.3.5. Estado de avance

Diseño: 100 %. El dimensionamiento está completo y el módulo step-down está incluido en el presupuesto.

Implementación física: pendiente (sujeta a la adquisición del módulo).

Capítulo 4

Diagrama de Bloques del Hardware

4.1. Decisión de diseño: un diagrama jerárquico en tres niveles

El hardware del sistema SmartRoad Rover integra cinco bloques funcionales (alimentación, sensado, procesamiento, actuación y comunicación), cada uno con señales, niveles de voltaje y protocolos propios. Presentar todo el detalle en un único diagrama produciría un esquema ilegible a escala de página. Por esta razón, y siguiendo la práctica estándar en diseño de sistemas embebidos, se adopta una **jerarquía de tres diagramas**:

- **Diagrama 1 — Vista general (top-level):** muestra los cinco bloques principales y sus relaciones. No incluye pines ni señales individuales. Es el mapa unificador del sistema.
- **Diagrama 2 — Bloque de sensado y acondicionamiento:** detalla los tres sensores HC-SR04, sus circuitos de adaptación de nivel lógico (divisores de tensión), el módulo GPS y la asignación de pines GPIO del Pico W.
- **Diagrama 3 — Bloque de actuación y alimentación:** detalla el driver TB6612FNG, los dos motores DC, el sistema de alimentación con sus dos dominios de voltaje y la distribución de tierra común.

Esta estructura cumple con el requerimiento del enunciado: existe un diagrama que relaciona los demás (Diagrama 1) y cada diagrama de detalle está explicado textualmente en las secciones siguientes.

4.2. Asignación de pines GPIO del Pico W

Antes de describir los diagramas, se fija la asignación definitiva de pines del microcontrolador. Esta tabla es la referencia para el firmware y para el montaje en protoboard:

Señal	GPIO	Pin físico	Dirección / Nota
TRIG sensor frontal	GP0	1	Salida digital
ECHO sensor frontal	GP1	2	Entrada — viene del divisor de tensión
TRIG sensor izquierdo	GP2	4	Salida digital
ECHO sensor izquierdo	GP3	5	Entrada — viene del divisor de tensión
TRIG sensor derecho	GP4	6	Salida digital
ECHO sensor derecho	GP5	7	Entrada — viene del divisor de tensión
GPS TX → Pico RX	GP9	12	UART1 RX
GPS RX → Pico TX	GP8	11	UART1 TX
IN1 Motor A (izq.)	GP10	14	Salida digital — dirección
IN2 Motor A (izq.)	GP11	15	Salida digital — dirección
PWM Motor A (izq.)	GP12	16	Salida PWM — velocidad
IN1 Motor B (der.)	GP13	17	Salida digital — dirección
IN2 Motor B (der.)	GP14	19	Salida digital — dirección
PWM Motor B (der.)	GP15	20	Salida PWM — velocidad
STBY TB6612FNG	GP16	21	Salida digital — enable global

Cuadro 4.1: Asignación definitiva de pines GPIO del Raspberry Pi Pico W

4.3. Diagrama 1 — Vista general del sistema

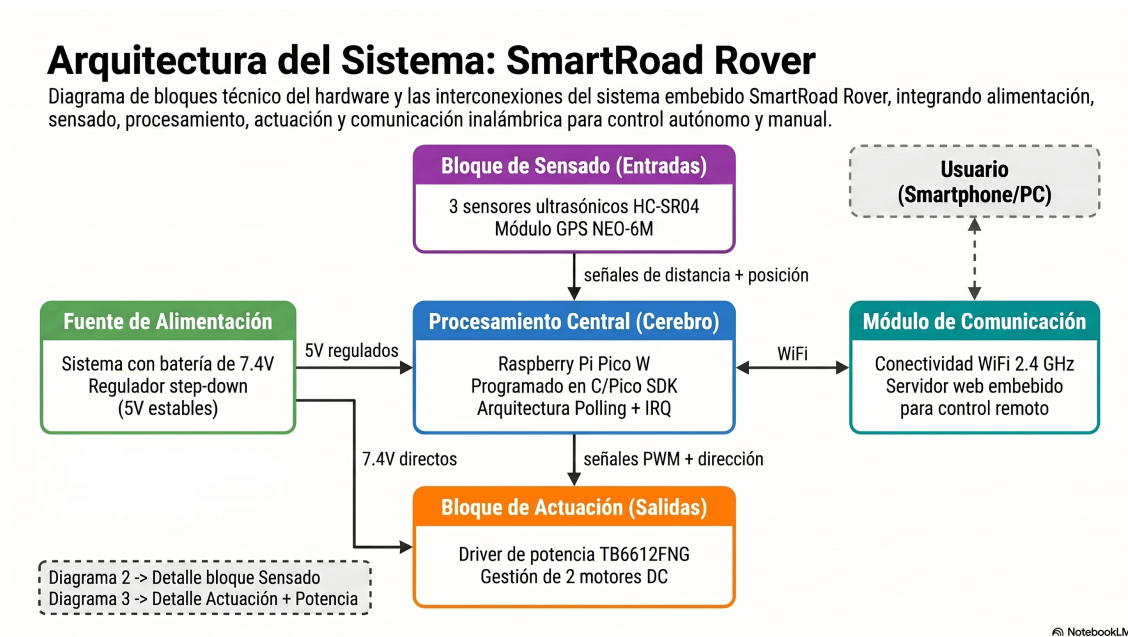


Figura 4.1: Diagrama de bloques top-level del sistema SmartRoad Rover

Descripción: el sistema se organiza alrededor del bloque de procesamiento central (Raspberry Pi Pico W). Este recibe señales del bloque de sensado (sensores ultrasónicos y GPS), genera señales de control hacia el bloque de actuación (driver de motores y motores DC), se comunica bidireccionalmente con el usuario a través del bloque de comunicación WiFi, y recibe energía del bloque de alimentación.

Las relaciones entre bloques son:

- **Alimentación → Procesamiento:** suministra 5 V regulados al pin VSYS del Pico W.
- **Alimentación → Actuación:** suministra 7.4 V directos al pin VM del driver TB6612FNG.
- **Sensado → Procesamiento:** entrega señales digitales de distancia (ECHO adaptado) y tramas NMEA del GPS vía UART.
- **Procesamiento → Actuación:** genera señales PWM de velocidad y señales digitales de dirección hacia el driver.
- **Procesamiento - Comunicación:** intercambio bidireccional vía WiFi 2.4 GHz para recepción de comandos del usuario y envío de estado del sistema.

4.4. Diagrama 2 — Bloque de sensado y acondicionamiento

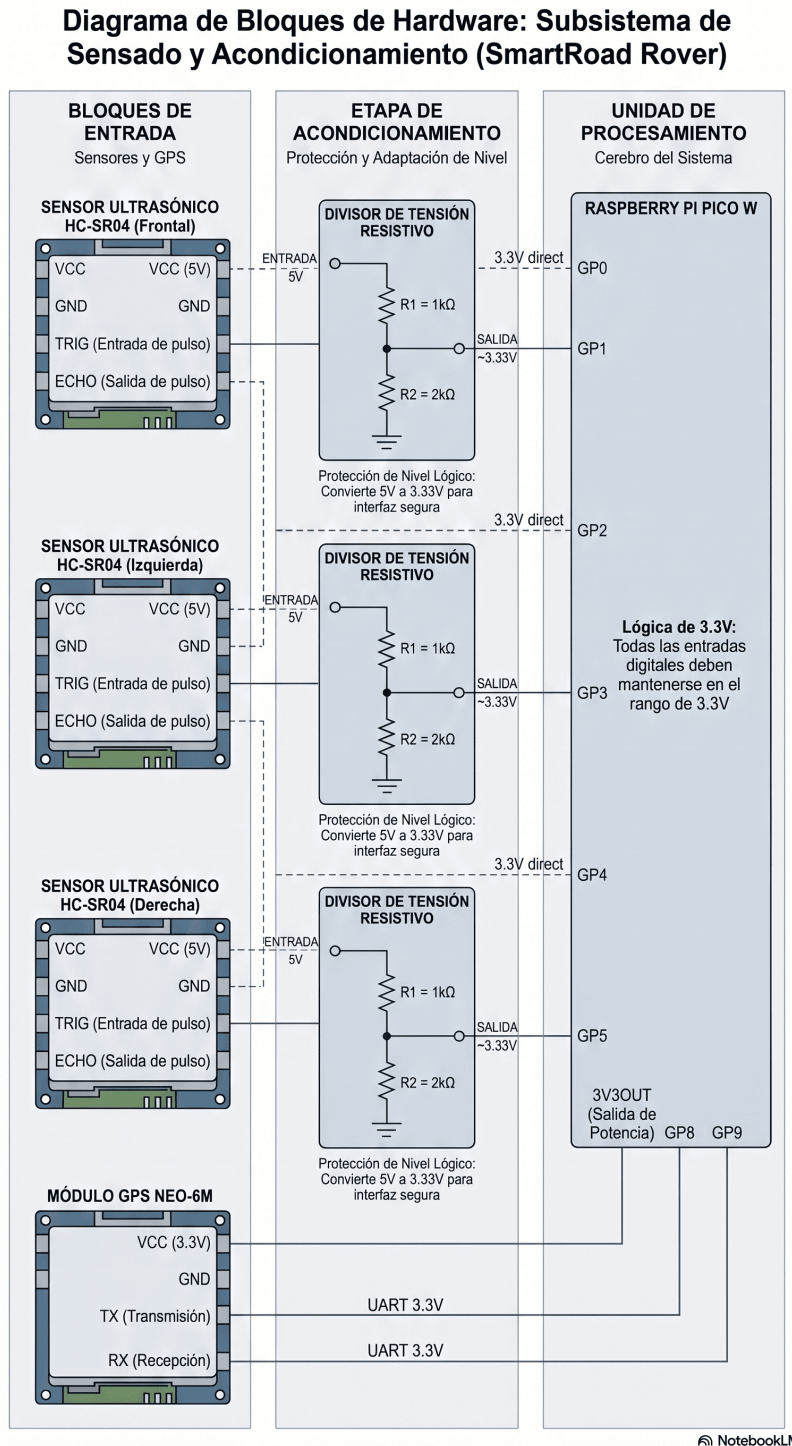


Figura 4.2: Diagrama de bloques del bloque de sensado — HC-SR04 ×3, divisores de tensión y módulo GPS

Descripción: el diagrama muestra cómo fluyen las señales desde los cuatro sensores hacia el Pico W y qué transformaciones ocurren en el camino. Cada HC-SR04 genera dos señales de distinta naturaleza: una de salida del Pico W (TRIG) que viaja directamente al sensor, y una de entrada (ECHO) que debe pasar por el circuito de acondicionamiento de nivel lógico detallado en el Capítulo 3 antes de llegar al GPIO. Este acondicionamiento es lo que hace el bloque de sensado más complejo que una simple conexión directa y justifica su diagrama propio.

Desde el punto de vista del sistema, lo que el Pico W recibe de este bloque son **seis señales digitales independientes** (un TRIG de salida y un ECHO de entrada por cada uno de los tres sensores), todas a nivel lógico de 3.3 V, y **dos señales UART** del GPS (TX y RX). El procesamiento central no necesita conocer los detalles del acondicionamiento; solo ve señales lógicas válidas en sus GPIO.

El GPS introduce un flujo de datos de naturaleza distinta: tramas NMEA seriales a 9600 bps que el Pico W recibe por UART1 y procesa mediante DMA (descrito en el Capítulo 5). Su alimentación proviene del pin 3V3OUT del Pico W, lo que lo hace dependiente del encendido del microcontrolador.

4.5. Diagrama 3 — Bloque de actuación y sistema de alimentación

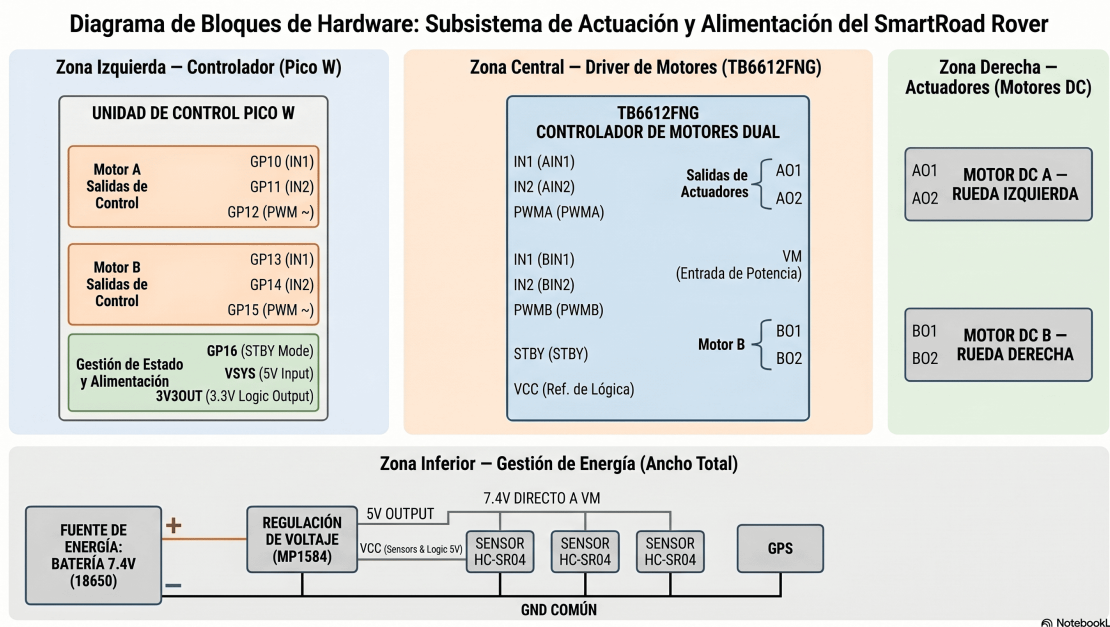


Figura 4.3: Diagrama de bloques del bloque de actuación y sistema de alimentación

Descripción: este diagrama muestra dos aspectos del sistema que están íntimamente relacionados: cómo el Pico W controla el movimiento del rover, y cómo la energía de la batería se distribuye a cada componente. Se presentan juntos porque el sistema de alimentación

(detallado en el Capítulo 3) determina directamente los voltajes disponibles en cada nodo del bloque de actuación.

Desde el punto de vista del control, el Pico W ejerce autoridad completa sobre el movimiento mediante **siete señales de salida** hacia el TB6612FNG: tres por canal de motor (dirección A, dirección B y PWM de velocidad) más la señal STBY que habilita o deshabilita globalmente el driver. Esta separación entre señales de dirección (digitales simples) y señal de velocidad (PWM) permite control independiente y preciso de cada rueda, lo que es la base de las maniobras de giro del rover.

Desde el punto de vista energético, el diagrama evidencia la separación de dominios de potencia: los motores operan directamente con la tensión de batería sin regulación, mientras que toda la lógica opera desde el dominio regulado de 5 V. Esta separación protege al microcontrolador de los picos de corriente que generan los motores al arrancar o frenar bruscamente.

Capítulo 5

Arquitectura de Software

5.1. Visión general y decisiones de diseño

Nota sobre el estado del firmware: el código presentado en este capítulo constituye la **arquitectura de firmware diseñada**. Los fragmentos mostrados representan la implementación planificada de cada módulo, resultado de las decisiones de diseño tomadas durante las Semanas 1 y 2 del plan de trabajo. La compilación, integración y prueba sobre el hardware físico corresponden a las Semanas 4 y 5, cuyo avance actual se describe en el Capítulo 1.

El firmware se implementa en **C con el Pico SDK oficial** de Raspberry Pi. La arquitectura adopta el paradigma **Polling + IRQ**: las operaciones de tiempo crítico (medición de eco ultrasónico, recepción GPS) se manejan mediante interrupciones de hardware; la lógica de navegación, la gestión de modos y el servidor web operan mediante polling en el loop principal. No se usa RTOS. Los periféricos del RP2040 (GPIO IRQ, PWM hardware, UART, DMA, timers de hardware) se explotan directamente para lograr comportamiento determinista sin bloquear el CPU.

5.2. Estructura modular del firmware

El código se organiza en módulos independientes, cada uno con su archivo `.c` y su cabecera `.h`:

Módulo	Responsabilidad
main.c	Loop principal, inicialización del sistema, despacho de la FSM
sensors.c/.h	Disparo TRIG (polling), ISR ECHO, cálculo de distancia, filtrado por mediana
motors.c/.h	PWM de hardware, control de dirección, abstracción del TB6612FNG
navigation.c/.h	Algoritmo de evasión autónoma, selección de dirección de giro
web_server.c/.h	Servidor HTTP embebido (lwIP), parsing de comandos, watchdog WiFi
gps.c/.h	UART1 + DMA, parsing de tramas NMEA, cálculo de rumbo, watchdog GPS
fsm.c/.h	FSM principal: 5 estados, transiciones nominales y de contingencia

Cuadro 5.1: Estructura modular del firmware SmartRoad Rover

Cada módulo es compilable y testeable de forma independiente. La variable de estado de la FSM es `volatile` para garantizar visibilidad entre el contexto de IRQ y el loop principal.

5.3. Máquina de estados finita (FSM) — fsm.c

La FSM principal tiene **5 estados**. Solo un estado está activo en cualquier instante. Se implementa con `enum` y `switch-case` en el loop principal:

```

1  typedef enum {
2      STATE_INIT,
3      STATE_MANUAL,
4      STATE_AUTO_REACTIVE,
5      STATE_AUTO_GPS,
6      STATE_SAFE_STOP
7  } system_state_t;
8
9  volatile system_state_t current_state = STATE_INIT;
10
11 int main(void) {
12     system_init();
13     while (true) {
14         switch (current_state) {
15             case STATE_INIT:          fsm_init();          break;
16             case STATE_MANUAL:        fsm_manual();         break;
17             case STATE_AUTO_REACTIVE: fsm_auto_reactive();  break;
18             case STATE_AUTO_GPS:      fsm_auto_gps();       break;
19             case STATE_SAFE_STOP:     fsm_safe_stop();       break;
20         }
21     }

```

Transiciones nominales: INIT $\rightarrow$ MANUAL / AUTO_REACTIVE / AUTO_GPS según comando del usuario vía web. Cualquier estado $\leftrightarrow$ cualquier otro por comando web.

Transiciones de contingencia:

- Pérdida de WiFi > 500 ms $\rightarrow$ STATE_SAFE_STOP (RNF-5)
- Pérdida de señal GPS > 3 s $\rightarrow$ STATE_SAFE_STOP $\rightarrow$ STATE_AUTO_REACTIVE (RNF-6)
- Reconexión o comando de usuario $\rightarrow$ salida de STATE_SAFE_STOP

La figura 5.1 ilustra el diagrama de transiciones completo de la FSM principal. Las flechas indican las condiciones que disparan cada cambio de estado, incluyendo los casos de contingencia: pérdida de conexión WiFi en modo manual y pérdida de señal GPS en modo de navegación, ambos con transición automática hacia STATE_SAFE_STOP.

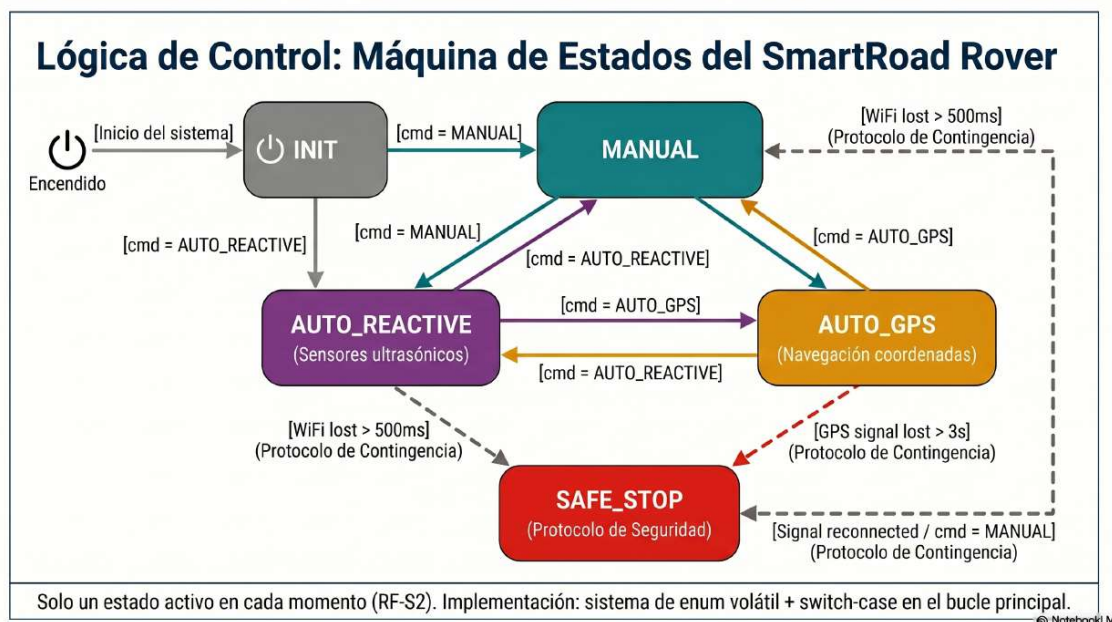


Figura 5.1: Diagrama FSM principal del sistema SmartRoad Rover — Realizado con NotebookLM

5.4. Módulo de sensores — ISR y filtrado (sensors.c)

La medición de distancia con HC-SR04 exige medir el tiempo de vuelo del pulso ECHO con precisión de microsegundos. Se usan **interrupciones GPIO en ambos flancos**:

```
1 void echo_irq_handler(uint gpio, uint32_t events) {
2     uint32_t now = time_us_32();
3     uint8_t idx = gpio_to_sensor_index(gpio);
4
5     if (events & GPIO_IRQ_EDGE_RISE) {
6         sensor_data[idx].t_rise = now;
7     }
8     if (events & GPIO_IRQ_EDGE_FALL) {
9         uint32_t pulse_us = now - sensor_data[idx].t_rise;
10        sensor_data[idx].distance_cm = (pulse_us * 17) / 1000;
11        sensor_data[idx].valid = true;
12    }
13 }
```

Filtrado por mediana (RNF-9): cada canal mantiene un buffer circular de 3 lecturas. La distancia reportada al resto del sistema es la mediana de las últimas 3 mediciones válidas, descartando picos por reflexiones espurias sin introducir latencia significativa.

Disparo de TRIG (polling): síncrono desde el loop principal. Se usa `busy_wait_us(10)` del SDK, determinista y sin bloquear el sistema de interrupciones:

```
1 void sensors_trigger_all(void) {
2     const uint trig_pins[] = {GP0, GP2, GP4};
3     for (int i = 0; i < NUM_SENSORS; i++) {
4         gpio_put(trig_pins[i], 1);
5         busy_wait_us(10);
6         gpio_put(trig_pins[i], 0);
7     }
8 }
```


5.5. Módulo de motores — PWM de hardware (motors.c)

Se usa **PWM de hardware** del RP2040 a 20 kHz (inaudible, adecuado para motores DC pequeños). Cada motor tiene un slice PWM independiente:

```
1 void motors_init(void) {
2     gpio_set_function(GP12, GPIO_FUNC_PWM); /* Motor A */
3     gpio_set_function(GP15, GPIO_FUNC_PWM); /* Motor B */
4     uint slice_a = pwm_gpio_to_slice_num(GP12);
5     uint slice_b = pwm_gpio_to_slice_num(GP15);
6     pwm_config cfg = pwm_get_default_config();
7     pwm_config_set_clkdiv(&cfg, 6.25f); /* 20 kHz */
8     pwm_config_set_wrap(&cfg, 999); /* resolution 0-999 */
9     pwm_init(slice_a, &cfg, true);
10    pwm_init(slice_b, &cfg, true);
11 }
12
13 void motors_set(motor_dir_t dir_a, uint16_t spd_a,
14                motor_dir_t dir_b, uint16_t spd_b) {
15     gpio_put(GP10, (dir_a == FORWARD) ? 1 : 0);
16     gpio_put(GP11, (dir_a == FORWARD) ? 0 : 1);
17     gpio_put(GP13, (dir_b == FORWARD) ? 1 : 0);
18     gpio_put(GP14, (dir_b == FORWARD) ? 0 : 1);
19     pwm_set_gpio_level(GP12, spd_a);
20     pwm_set_gpio_level(GP15, spd_b);
21 }
```

5.6. Módulo de navegación autónoma (navigation.c)

Implementa el algoritmo de evasión. Se ejecuta en el loop principal consumiendo datos ya calculados por la ISR, sin bloquear el sistema de interrupciones.

Caso de contingencia: los tres sensores detectan obstáculo simultáneamente. Si los tres sensores reportan distancia menor al umbral, el rover ejecuta una maniobra de retroceso durante 400 ms seguida de un giro a la derecha por defecto durante 300 ms, antes de retomar el ciclo. Esta decisión es determinista y garantiza que el sistema nunca quede bloqueado indefinidamente:

```
1 void navigation_step(void) {
2     uint16_t front = sensors_get_filtered(FRONT);
3     uint16_t left  = sensors_get_filtered(LEFT);
4     uint16_t right = sensors_get_filtered(RIGHT);
5
6     /* Caso especial: los tres sensores detectan obstaculo */
7     if (front < OBSTACLE_THRESHOLD_CM &&
8         left  < OBSTACLE_THRESHOLD_CM &&
9         right < OBSTACLE_THRESHOLD_CM) {
10        motors_set(REVERSE, SPEED_TURN, REVERSE, SPEED_TURN);
11        busy_wait_ms(400);
12        motors_set(FORWARD, SPEED_TURN, REVERSE, SPEED_TURN);
13        busy_wait_ms(300);
14        return;
15    }
16
17    if (front > OBSTACLE_THRESHOLD_CM) {
18        motors_set(FORWARD, SPEED_FULL, FORWARD, SPEED_FULL);
19    } else {
20        motors_set(STOP, 0, STOP, 0);
21        busy_wait_ms(50);
22        if (left >= right) {
23            motors_set(REVERSE, SPEED_TURN, FORWARD, SPEED_TURN);
24        } else {
25            motors_set(FORWARD, SPEED_TURN, REVERSE, SPEED_TURN);
26        }
27        busy_wait_ms(300);
28    }
29 }
```

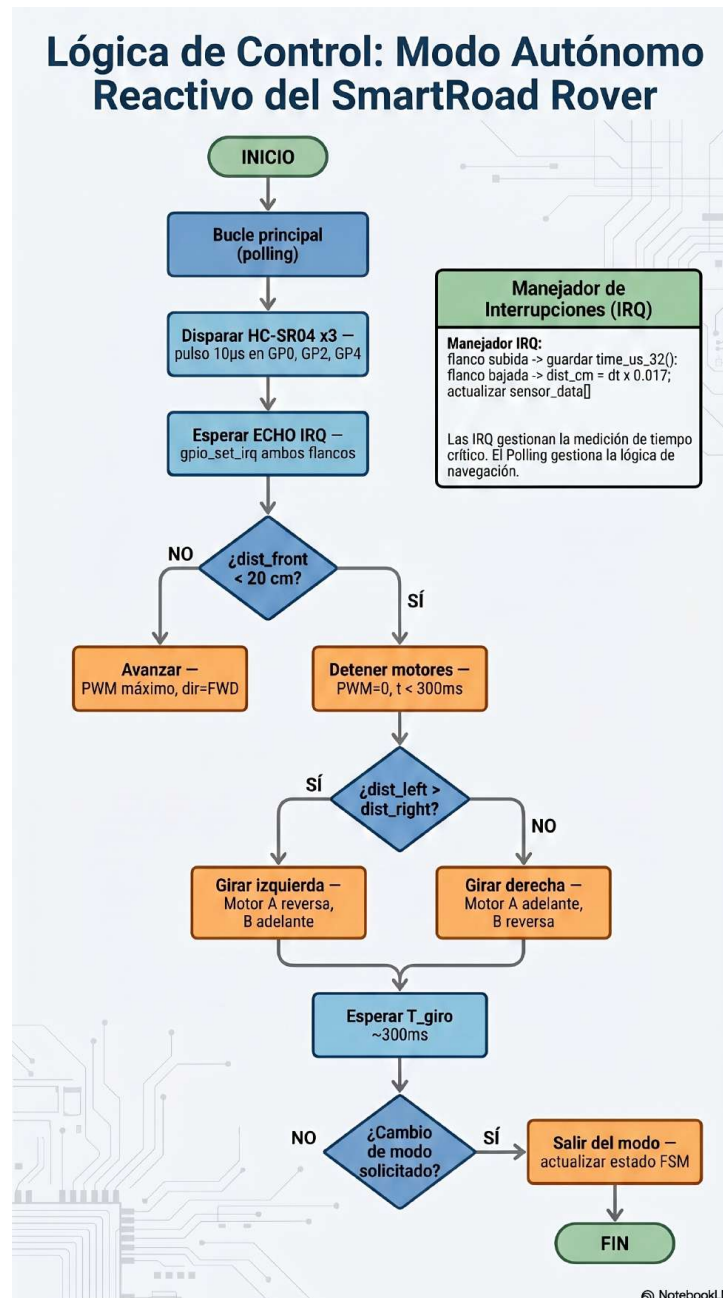


Figura 5.2: Diagrama de flujo — modo autónomo reactivo (Polling + IRQ) — Realizado con NotebookLM

5.7. Módulo GPS con DMA (gps.c) — Reporte DMA

Se usa **1 canal DMA** para recepción de tramas NMEA por UART1. El DMA transfiere bytes del registro UART1_DR a un buffer circular en RAM mediante el DREQ DREQ_UART1_RX, sin intervención del CPU. Cuando la transferencia completa, DMA_IRQ_0 señala al loop principal que hay una trama lista:

```

1 void gps_init(void) {
2     uart_init(UART_GPS, GPS_BAUD);
3     gpio_set_function(GP8, GPIO_FUNC_UART); /* TX */
4     gpio_set_function(GP9, GPIO_FUNC_UART); /* RX */
5
6     dma_channel = dma_claim_unused_channel(true);
7     dma_channel_config cfg =
8         dma_channel_get_default_config(dma_channel);
9     channel_config_set_transfer_data_size(&cfg, DMA_SIZE_8);
10    channel_config_set_read_increment(&cfg, false);
11    channel_config_set_write_increment(&cfg, true);
12    channel_config_set_dreq(&cfg, DREQ_UART1_RX);
13
14    dma_channel_configure(dma_channel, &cfg,
15        gps_buffer,
16        &uart_get_hw(UART_GPS)->dr,
17        GPS_BUF_SIZE, true);
18
19    dma_channel_set_irq0_enabled(dma_channel, true);
20    irq_set_exclusive_handler(DMA_IRQ_0, gps_dma_irq_handler);
21    irq_set_enabled(DMA_IRQ_0, true);
22 }

```

Watchdog GPS (RNF-6): un repeating timer de 1 s verifica si han transcurrido más de 3 segundos sin trama válida. De ser así, transita a STATE_SAFE_STOP:

```

1 bool gps_signal_valid(void) {
2     return (time_us_64() - gps_last_valid_us) < GPS_TIMEOUT_US;
3 }

```

5.8. Módulo servidor web — Watchdog WiFi (web_server.c)

Servidor HTTP embebido sobre el stack **lwIP** del Pico W SDK. El parsing de comandos es no bloqueante mediante callbacks de lwIP. Un repeating timer de 100 ms verifica el timeout del último comando recibido (RNF-5):

```

1 bool wifi_watchdog_callback(repeating_timer_t *t) {
2     if ((time_us_64() - last_cmd_us) > WIFI_TIMEOUT_US) {
3         motors_set(STOP, 0, STOP, 0);
4         current_state = STATE_SAFE_STOP;
5     }
6     return true;
7 }

```

5.9. Inventario de interrupciones

Fuente IRQ	Tipo	Acción en el handler
GPIO GP1 — ECHO frontal	Flanco ↑↓	Captura tiempo, calcula distancia, actualiza buffer
GPIO GP3 — ECHO izquierdo	Flanco ↑↓	Ídem
GPIO GP5 — ECHO derecho	Flanco ↑↓	Ídem
DMA_IRQ_0	Transferencia completa	Señaliza trama GPS disponible, recarga DMA
Repeating timer 100 ms	Periódica	Watchdog WiFi — verifica timeout HTTP
Repeating timer 1 s	Periódica	Watchdog GPS — verifica timeout señal

Cuadro 5.2: Inventario de interrupciones del firmware

5.10. Librerías del Pico SDK utilizadas

Librería	Uso
pico/stdlib.h	Funciones base, stdio, delays
hardware/gpio.h	Configuración de pines e IRQ GPIO
hardware/pwm.h	PWM de hardware para motores
hardware/uart.h	Comunicación UART con GPS
hardware/dma.h	Canal DMA para recepción GPS
hardware/timer.h	time_us_32/64(), repeating timers
pico/cyw43_arch.h	WiFi del chip CYW43439 del Pico W
lwip/tcp.h	Stack TCP/IP para servidor HTTP embebido

Cuadro 5.3: Librerías del Pico SDK empleadas en el firmware

Capítulo 6

Manejo del Repositorio

6.1. Repositorio del proyecto

El repositorio del proyecto SmartRoad Rover se encuentra disponible en:

<https://github.com/juanforozco/smartroad-rover>

El repositorio ha estado activo desde el inicio del proceso de definición del proyecto y ha sido actualizado de forma continua a lo largo de cada entregable, reflejando en todo momento el estado real del avance.

6.2. Estructura de carpetas

El repositorio está organizado en cuatro carpetas principales, cada una con su propio README.md:

Carpeta	Contenido
docs/	Documentos formales de entrega organizados por entregable en subcarpetas (<code>entregable1/</code> , <code>entregable2/</code> , <code>entregable3/</code>). Cada subcarpeta contiene el archivo fuente LaTeX y los PDFs generados.
firmware/	Código fuente del microcontrolador en C con el Pico SDK. Contendrá los archivos <code>.c</code> y <code>.h</code> de cada módulo (<code>sensors</code> , <code>motors</code> , <code>navigation</code> , <code>web_server</code> , <code>gps</code> , <code>fsm</code>) junto con el <code>CMakeLists.txt</code> .
hardware/	Documentación del hardware: esquemático del circuito y tabla de asignación de pines del Pico W.
images/	Imágenes de la documentación: diagramas de bloques, FSM, diagramas de flujo y fotos del prototipo.

Cuadro 6.1: Estructura de carpetas del repositorio

6.3. Estrategia de ramas

El repositorio trabaja con dos ramas principales:

- **main:** rama de producción. Solo recibe commits cuando un entregable o funcionalidad está completamente terminada y verificada. Refleja el estado oficial y estable del proyecto en todo momento.
- **dev:** rama de desarrollo activo. Aquí se trabajan los cambios, se agregan documentos en progreso y se desarrolla el firmware antes de integrarlo a **main**. Una vez verificado, se realiza un merge a **main** mediante pull request.

Esta estrategia garantiza que **main** siempre tenga un estado coherente y presentable, mientras que **dev** refleja el trabajo en curso sin afectar la versión oficial.

6.4. Convención de mensajes de commit

Se usa una convención de prefijos para que el historial sea legible y trazable:

Prefijo	Uso
docs:	Cambios en documentación y entregables
firmware:	Código del microcontrolador
hardware:	Esquemáticos y asignación de pines
fix:	Correcciones de cualquier tipo
refactor:	Reorganización sin cambio de funcionalidad

Cuadro 6.2: Convención de mensajes de commit

Ejemplos del historial actual: *"docs: agregar capítulo 5 - arquitectura de software"*, *"docs: correcciones ronda 2 basadas en retroalimentación docente"*, *"docs: crear estructura entregable 3"*.

6.5. Integración con VS Code y Pico SDK

Durante la fase de implementación del firmware, el flujo de trabajo será el siguiente: el repositorio se clona localmente, VS Code abre directamente la carpeta del repositorio como espacio de trabajo, y la extensión oficial de Raspberry Pi Pico se configura apuntando a esa misma carpeta. Esto permite editar el código, compilar con el Pico SDK y realizar commits y push a GitHub, todo desde VS Code sin necesidad de alternar entre herramientas. El archivo `CMakeLists.txt` del proyecto estará en `firmware/` y cada módulo tendrá su propio archivo fuente, coherente con la arquitectura modular descrita en el Capítulo 5.

6.6. Estado actual del repositorio

El repositorio refleja el avance real del proyecto en cada una de sus secciones:

- `docs/`: documentos de los tres entregables subidos y organizados por subcarpeta.
- `images/`: diagramas de bloques (top-level, sensado, actuación), FSM principal y diagrama de flujo del modo autónomo reactivo.
- `firmware/`: carpeta preparada con su `README.md` describiendo la estructura modular planificada, lista para recibir el código de implementación.
- `hardware/`: esquemático y tabla de pines documentados.
- `README.md` principal: actualizado con descripción completa del sistema, modos de operación, requisitos, escenario de pruebas y presupuesto vigente (252.000 COP).

Bibliografía

- [1] Raspberry Pi Foundation. *Raspberry Pi Pico C/C++ SDK*. <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>
- [2] Raspberry Pi Foundation. *Raspberry Pi Pico W Datasheet*.
- [3] *HC-SR04 Ultrasonic Sensor Datasheet*.
- [4] Toshiba. *TB6612FNG Motor Driver Datasheet*.
- [5] u-blox. *NEO-6M GPS Module Datasheet*.
- [6] Michael Barr. *Embedded Systems Engineering*. O'Reilly.